

Hi-Z 오클루전 컬링 결과를 활용한 가시성 기반 애니메이션 갱신 기법

장명운[○], 이상훈, 이종진, 정내훈

한국공학대학교 게임공학과

wkdauddds9@gmail.com, asm1133@naver.com, ljj3640@gmail.com,

nhjung@tukorea.ac.kr

A Visibility-based Animation Update Method Using Hi-Z Occlusion Culling

Myeong-Woon Jang[○], Sang-Hoon Woo, Jong-Jin Lee, Nai-Hoon Jung
Tech University of Korea

요 약

현대의 실시간 렌더링 분야에서는 Hi-Z 기반 오클루전 컬링과 같은 GPU 중심의 가시성 판별 기법이 널리 활용되고 있다. 그러나 이러한 가시성 정보는 주로 렌더링 단계 최적화에만 집중적으로 활용되며, 복잡한 계어로 인해 CPU에서 수행되어야 하는 애니메이션 갱신 비용 절감에는 충분히 활용되지 않고 있다. 본 연구는 GPU에서 판정한 가시성 정보를 활용하여, 보이지 않는 객체의 애니메이션 업데이트를 선택적으로 생략하는 기법을 제안한다. 제안 기법은 GPU에서 계산된 가시성 정보를 이전 프레임 기반으로 비동기적으로 활용하여, CPU-GPU 간 동기화 병목 없이 CPU 애니메이션 시스템에 반영할 수 있다. 이를 통해 역기구학이나 래그돌 물리 등 CPU 기반 처리가 요구되어 병목이 발생하는 애니메이션 환경에서, 애니메이션 연산량을 전체 객체 수가 아닌 실제 가시 객체 수에 비례하도록 감소시켜 성능을 향상한다.

1. 서 론

현대 실시간 렌더링 분야에서는 고품질 메시와 복잡한 조명 및 물리 효과가 적용된 장면을 높은 프레임 재생률로 처리하는 것을 목표로 한다. 그러나 현실적으로 장면의 모든 대상을 높은 품질로 렌더링하면서 충분한 프레임 재생률을 달성하는 것은 불가능하다.

이러한 성능 문제를 완화하기 위해, 렌더링 단계에서는 시각적 영향이 적은 연산을 줄이거나 생략하는 다양한 최적화 기법이 널리 사용된다. 대표적으로, LOD(Level of Detail) 기법은 객체의 거리나 화면 기여도에 따라 연산을 간략화하며,

시야 절두체 컬링 및 오클루전 컬링은 화면에 기여하지 않는 객체의 렌더링을 생략한다. 특히 Hi-Z 오클루전 컬링은 GPU에서 효율적으로 가시성을 판별할 수 있는 대표적인 기법이다.

그러나 이러한 가시성 기반 최적화는 주로 장면을 그리는 단계에 한정되어 활용되며, 애니메이션 갱신에는 충분히 반영되지 못하고 있다. CPU 기반 컬링 기법은 애니메이션 갱신에 직접적으로 활용될 수 있지만, GPU에서 수행되는 오클루전 컬링의 경우 CPU와 GPU의 비동기적 실행 특성으로 인해 그 결과를 즉각적으로 활용하기 어렵다. 그 결과, 화면에 기여하지 않는 객체들에 대해서도 불필요한 애니메이션 연산이 수행되는 문제가 발생한다.

본 연구에서는 GPU에서 수행되는 Hi-Z 오클루전 컬링 결과를 비동기적으로 CPU에서 활용하여 애니메이션 갱신 최적화에 반영하는 방법을 제안한다. 제안하는 방법은 CPU-GPU 병렬성을 잃지 않으면서 가시성 정보를 최대한 안정적으로 활용할 수 있도록 설계되었으며, 이를 통해 렌더링 품질을 유지하면서 CPU 기반 애니메이션 연산 비용을 효과적으로 절감할 수 있다.

2. 관련 연구

Bryan Dudash(2007)는 애니메이션 프레임 데이터를 사전에 텍스처 형태로 저장하고, 정점 셰이더에서 이를 참조하여 스키닝을 수행하는 스키닝 메시 대상 인스턴싱 기법을 제안하였다[1]. 이 방법은 다수의 애니메이션 객체를 효율적으로 렌더링할 수 있으나, 인스턴스별로 서로 다른 애니메이션 상태를 요구하는 경우, 예를 들어 실시간 보간, 역기구학, 또는 물리 기반 애니메이션과 같은 상황에는 적용이 제한된다.

Ned Greene, Michael Kass, Gavin Miller(1993)은 장면을 팔진 트리로 분할하여 화면에 보이지 않는 노드 속의 폴리곤들을 컬링하는 데 깊이 버퍼를 활용할 수 있도록 계층적으로 깊이 버퍼를 구성하는 Hi-Z 오클루전 컬링 기법을 소개한다[2]. 해당 연구에서 소개한 Hi-Z 오클루전 컬링 기법은 여러 형태로 응용되어 현대의 다양한 GPU 기반 컬링 기법의 기반이 되었다.

Hansong Zhang, Dinesh Manocha, Tom Hudson, Kenneth E. Hoff III(1997)는 장면을 팔진 트리로 분할하는 대신 렌더링 되는 객체들에 대해 바운딩 볼륨 계층 구조를 구성하여 객체 단위로 컬링될 수 있도록 하였으며, 시야 절두체 컬링과 조합하여 렌더링에 효과적으로 통합하는 방법을 제시하였다[3]. 본 연구의 컬링 파이프라인은 이 방법에 기반한다.

Ulrich Haar, Sebastian Aaltonen(2015)은 최신 그래픽스 API를 통해 GPU에서 컬링과 그리기

명령 생성을 수행하고, CPU-GPU 간 동기화를 최소화하는 렌더링 구조를 제안하였다[4]. 그러나 이러한 접근은 그리기 명령 생성 과정에 대해서만 유효하며 애니메이션 갱신과 같은 CPU 중심 연산에 대한 최적화는 고려되지 않는다. 본 연구에서는 GPU에서 생성된 가시성 정보를 CPU 애니메이션 시스템에 반영함으로써 애니메이션 갱신을 효율적으로 수행하는 방법을 제안한다.

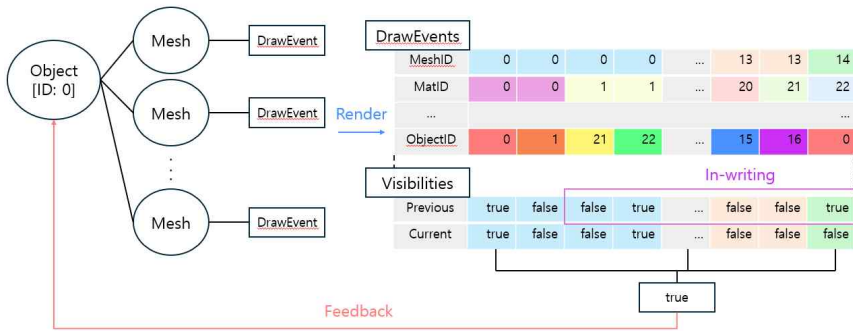
3. 가시성 기반 애니메이션 갱신

3.1. 시스템 개요

각 객체는 복수의 메시와 재질의 조합으로 구성되며, 렌더링 시 각 메시-재질 쌍에 대해 하나의 그리기 이벤트가 생성된다. 이때, 시야 절두체 컬링을 통해 비가시로 판정된 객체는 그리기 이벤트를 생성하지 않는다. 생성된 그리기 이벤트에는 인스턴스의 변환 행렬, 메시 및 재질 정보, 객체 식별자가 포함된다.

제출된 그리기 이벤트들은 동일한 메시-재질 쌍을 공유하는 이벤트들이 연속적으로 배치되도록 정렬되며, 셰이더에서 반복적으로 요구되는 행렬 데이터가 미리 계산된다. 이후 전처리된 그리기 이벤트를 대상으로 Hi-Z 오클루전 컬링을 수행해 각 이벤트에 대응하는 32비트 정수형 가시성 플래그를 생성한다. 가시성은 1비트로 표현 가능하므로, 나머지 상위 비트에는 객체 식별자를 비트 단위 OR 연산을 통해 함께 저장한다.

이후 CPU에서 인스턴스 단위의 가시성 플래그들을 검사하며 객체 단위의 가시성을 결정하게 된다. 객체 단위의 가시성은 매번 비가시 판정으로 초기화된 뒤 구성 인스턴스가 하나라도 보이는 것으로 판별되면 가시 판정으로 갱신된다. CPU는 GPU와의 명시적인 동기화 없이 가시성 플래그 버퍼를 읽어들이며, 가시성 비트가 설정된 경우 상위 비트에 저장된 객체 식별자를 통해 해당 객체를 가시 상태로 판정한다. 이 방식은



[그림 1] 시스템 개요

CPU가 임의의 시점에서 버퍼를 읽더라도, 각 인스턴스의 최신 가시성 정보를 기반으로 판단이 이루어지도록 보장한다.

이렇게 수집된 객체 단위의 가시성 정보는 애니메이션 시스템으로 전달된다. 애니메이션 시스템은 현재 시야에 포함된 객체들만을 대상으로 거리 및 마지막 갱신 시점을 고려한 스케줄링을 수행하며, 주어진 시간 예산 내에서 최대한 많은 애니메이션을 갱신한다. 이를 통해 컬링 결과와 LOD가 자연스럽게 반영된 애니메이션 최적화가 이루어진다.

3.2. 동기화 회피 전략

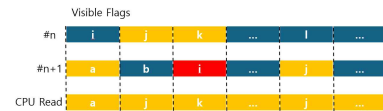
GPU와 CPU 간의 명시적인 동기화를 제거할 경우, CPU는 GPU가 아직 쓰기 작업을 완료하지 않은 가시성 버퍼를 읽게 될 수 있다. 이로 인해 특정 객체의 가시성은 해당 객체를 구성하는 인스턴스들의 이전 프레임과 현재 프레임의 가시성 정보가 혼합된 상태로 판정될 수 있다.

각 인스턴스의 이전 프레임 가시성, 현재 프레임 가시성, 그리고 영향을 미치는 객체의 가시성 판정 결과에 대해 총 8개의 조합이 존재하며, 정리하면 다음의 표와 같다.

[표 1] 가시성 판정 결과 조합

	#n	#n+1	최종
(1)	가시	가시	가시
(2)	비가시	가시	가시
(3)	가시	비가시	가시
(4)	비가시	비가시	가시
(5)	가시	가시	비가시
(6)	비가시	가시	비가시
(7)	가시	비가시	비가시
(8)	비가시	비가시	비가시

(1), (2), (7), (8)의 경우 올바르게 판정된 것이고, (3)의 경우는 시각적 결과에 영향을 미치지 않으므로 성능상 감내할 수 있다. (4)의 경우는 발생이 물리적으로 불가능하며, (6)의 경우는 한 프레임 늦게 가시성이 반영될 수 있으나 시각적 영향이 미미하다. 따라서 주의 깊게 봐야 할 것은 (5)의 경우이다.



[그림 2] 인스턴스 누락 상황

해당 상황은 인스턴스 데이터의 재배치로 인해 발생할 수 있다. 예를 들어, 특정 인스턴스 i 가 프레임 n 에서 인덱스 p , 프레임 $n+1$ 에서 인덱스 q 에 기록되며 $p \neq q$ 인 경우, CPU가 읽는 시점에 두 위치 모두에서 해당 인스턴스의 최신 가시성 정보가 반영되지 않을 수 있다. 이 경우,

[표 2] 성능 분석표

평균 애니메이션 갱신 횟수(회, mean ± SD)
평균 총 애니메이션 갱신 시간(ms, mean ± SD)

실제로는 두 프레임 동안 보였던 인스턴스들이 누락되어 객체 전체가 보이지 않는 것으로 잘못 판정된다.

이러한 문제를 방지하기 위해, GPU가 갱신하는 가시성 플래그 버퍼를 두 개의 슬롯을 갖는 링 버퍼로 구성한다. 그리고 CPU측 메모리에 슬롯별 컬링 대상 인스턴스 수를 별도로 저장한다. 각 프레임 렌더링 시 서로 다른 슬롯에 번갈아가며 가시성 판정 결과를 기록하며, 이를 통해 적어도 한 프레임에 대해 모든 인스턴스의 가시성 정보가 완전하게 보존되도록 한다. CPU는 두 슬롯을 모두 읽으며 가시성을 판별하는데, 별도로 기록된 슬롯별 인스턴스 수 이상으로는 읽지 않도록 한다. 이때, 두 슬롯에서 인스턴스가 중복될 수 있지만 가시성 판정 단계에서만 존재하는 중복이므로 성능을 유의미하게 감소시키지는 않는다.

3.3. 애니메이션 스케줄링

각 객체들의 애니메이션 우선순위는 다음과 같은 공식으로 갱신된다. (d : 카메라와 객체 사이의 거리, k_d : 거리 규모 상수, t : 마지막 갱신으로부터 경과한 시간, k_t : 시간 규모 상수, $priority$: 애니메이션 우선순위)

$$w_d = \frac{1}{1 + (d \times k_d)^2}$$

$$w_t = 1 + \frac{t}{t + k_t}$$

$$priority = w_d \times w_k$$

객체들의 애니메이션 우선 순위 계산이 끝나면, 애니메이션 시스템은 컬링된 객체들을 제외한 후 최대 힙 방식으로 우선순위 큐를 구성한다. 이후 우선순위 큐에서 객체들을 하나씩 꺼내 애니메이션을 갱신하다가 주어진 시간 예산을 다

사용하면 나머지 갱신은 생략한 뒤 종료한다. 이러한 스케줄링 방식을 통해 애니메이션에 대해서도 자연스럽게 컬링과 LOD가 적용된다.

4. 성능 분석

성능 분석은 AMD Ryzen 7 7800X3D 8-Core (4.20GHz) 프로세서, 32GB RAM, NVIDIA GeForce RTX 4070 Ti SUPER (16 GB), Windows 11 환경에서 샘플 장면들을 렌더링하는 것으로 수행되었다. 애니메이션 갱신에 Hi-Z 오cluirun 컬링과 시야 절두체 컬링을 모두 반영했을 때 평균 애니메이션 갱신 시간은 2.1ms, 시야 절두체 컬링만 반영했을 때 평균 애니메이션 갱신 시간은 4.3ms로, 제안 기법은 평균 애니메이션 갱신 시간에 있어 약 두 배 정도의 성능 향상을 보였다. 컬링 결과를 피드백하는 과정에서 302개의 객체에 대해 2,416 byte의 readback 데이터를 읽고 객체의 가시성 플래그와 비트 단위 OR 연산을 수행하게 되며, 이 과정의 수행 시간은 0.1ms를 넘지 않았다. 이는 CPU 기반 애니메이션이 주요 병목이 되는 환경에서 제안 기법이 큰 성능 향상을 달성할 수 있음을 시사한다.

5. 결론

5.1. 연구의 기여

본 연구에서는 GPU-CPU 동기화를 요구하지 않는 가시성 피드백 구조를 설계하여, 파이프라인 지연 없이 CPU로 가능한 한 최신의 가시성 정보를 활용할 수 있는 방법을 제시하였다. 해당 방법은 다른 GPU-Driven 렌더링

파이프라인과 연계될 수 있으며, 하드웨어 인스턴싱 단위를 유지하며 수행될 수 있다. 본 연구에서는 GPU에서 계산한 가시성 정보를 애니메이션 스케줄링에 반영하여 복잡한 애니메이션 연산을 효과적으로 줄일 수 있음을 보였다.

Real-Time Rendering Course, 2021.

본 연구는 2026년도 과학기술정보통신부 및 정보통신기획평가원의 'SW중심대학사업' 지원을 받아 수행되었음(2025-0-00050)

5.2. 향후 연구

본 연구는 메시, 재질 쌍 단위로 하드웨어 인스턴싱을 수행하는 렌더링 파이프라인에 기반하여 진행되었으나, 최신 GPU-Driven 렌더링 파이프라인은 삼각형 클러스터 단위로 추가 쿨링을 수행해 더 작은 단위로 LOD 적용 및 인스턴싱을 수행하는 경우가 많다[4][5]. 보다 세밀한 단위의 가시성을 역전파하는 것은 새로운 연구 과제가 될 수 있다.

또한 본 연구는 GPU 쿨링 결과를 애니메이션 갱신 최적화에 반영하는 것에 중점을 두었지만, 해당 결과를 물리 엔진, 게임의 전투 시스템 등 가시성을 효율적으로 활용할 수 있는 CPU 연산에 반영하는 것 역시 새로운 연구 과제가 될 수 있다.

참고문헌

- [1] B. Dudash. "Skinned Instancing.". In NVIDIA SDK 10, 2007.
- [2] N. Greene, M. Kass, and G. Miller. "Hierarchical Z-buffer visibility". In Proc. of ACM SIGGRAPH, pp 231 - 238, 1993.
- [3] H. Zhang, D. Manocha, T. Hudson, K. E. Hoff III, "Visibility Culling Using Hierarchical Occlusion Maps". In Proc. of ACM SIGGRAPH, pp 77-88. 1997.
- [4] U. Haar, S. Aaltonen, "GPU-Driven Rendering Pipelines". In SIGGRAPH 2015 Advances in Real-Time Rendering Course, 2015.
- [5] B. Karis, R. Stubbe, G. Wihlidal, "Nanite". In SIGGRAPH 2021 Advances in